Graphs and Theories: Simple Theories

Let's start generating some extremely simple theories. What does "extremely simple" mean? It means that the *vocabulary given to define theories* (by the meta-language) is simple. For example, we can assume that we have a single sort, that we only have function symbols with natural numbers as arity and co-arity exactly 1 and no relation symbols, i.e. not even equality in the meta-theory.

In other words, all we can define are very simple term languages. Of course, we'll want to add all sorts of features to our (meta) vocabulary. But, as soon as we do, the generation problem becomes progressively harder.

Another thing we want to do is to generate only *well-formed* and *well-typed* theories. It is perfectly fine if a theory has no models, but it can't be sheer nonsense. Quite a lot of technology will eventually be needed for that. Let us use *well-posed* to mean that is is well-formed, well-typed, well-scoped, and so on. This will be made clear for each language, but it is useful to have an all encompassing term to lighten the prose.

For pure term languages, this is easy. Which brings us to the next complication: size. We would like to have a reasonable definition of the size of a theory.

A naïve definition would essentially count the size of the resulting abstract syntax tree representing the theory. This is a reasonable enough heuristic in simple cases, but starts being quite odd for feature-full meta-languages. One way to think of it is: the printed presentation of certain theories are "optimized" for human-level comprehension but do not correspond to the full data of those theories. Quite a bit of information, required to insure well-posedness, is usually erased. Often a combination of *elaboration* and *inference* is used to "recover" such information in mechanized systems.

Thus we instead consider the following notion of size:

Definition 1. The size of a theory is the size of the Abstract Syntax Tree of the derivation that it is well-posed.

Underlying this definition is the (reasonable) assumption that things that resemble theories can be extracted from derivations.

For the simplest of theories, what is needed is quite simple:

- A source of unique names, to name each operation,
- A source of natural numbers, for each operation's arity.

In fact, we will not even use names in our internal representation, we will use something we know has decidable equality (here: the natural numbers again) but ask for some external entity to provide us translations to external names. We don't need to store the "name" as it can be recovered from its position.

We can already get some interesting information in the simplest cases (corresponding to single-sorted term theories with natural numbers as arity and co-arity 1). We can represent such theories as a list of natural numbers:

```
module Simple where

import Numeric.Natural

type Theory1 = [Natural]
```

Naively, the size would be just the sum of the arities, but this would ignore the size of the names themselves. The 'size' of an arity has to count all symbols, so 0 has size 1. Thus we get the following notion of length:

```
len :: Theory1 -> Integer
len ts = sum $ zipWith (+) [1..] (map ((+1).toInteger) ts)
```

where each name counts as its own length, and every arity counts also as the length of its representation.

The resulting representations of small size then are

- Size 0: `[]`
- Size 1: none
- Size 2: `[0]`
- Size 3: `[1]`
- Size 4: `[2]`
- Size 5: `[0,0]`, `[3]`
- Size 6: `[1,0]`, `[4]`
- Size 7: `[2,0]`, `[1,1]`, `[5]`
- Size 8: `[2,1]`, `[3,0]`, `[6]`
- Size 9: `[0,0,0]`, `[2,2]`, `[4,0]`, `[3,1]`, `[7]`

In English, we give a description and then, if it exists, a name frequently associated to such a theory.

- Size 0: No operations (Carrier)
- Size 2: A single constant (Pointed)
- Size 3: A single unary operation (Unar)
- Size 4: A single binary operation (Magma)
- Size 5:
  - Two constants (Bool)
  - A single ternary operation
  - A unary operation and a constant (Nat)
  - A single 4-ary operation

- A binary operation and a constant (Pointed Magma)
- Two unary operations
- A single 5-ary operation

- A binary operation and a unary operation
- A ternary operation and a constant
- A single 6-ary operation

- Three constants
- Two binary operations
- A 4-ary operation and a constant
- A 3-ary and a unary operation
- A single 8-ary operation

Why would `[0,0]` be size 5? Let's take a look at its expanded representation: `cons Z (cons Z nil)`. Lo and behold, 5 tokens!

It bears repeated that this *size* notion is not intrinsic. When we change our vocabulary, we'll see that this theory will re-appear but at a different size. This is because the encoding needs to change and the encoding needs to accounts for the increased power in the meta-language.